数据挖掘与大数据实验一

实验目的

实验目标：关联规则挖掘并行化实践

实验内容

任务一

1、使用python实现单机环境下的Apriori算法，不得调用已有的Apriori算法软件包

任务二

2、在mapreduce环境下设计并实现并行Apriori算法，要求采用Java语言实现

任务三

3、实现随机化采样算法SON算法进行关联规则挖掘，其中数据随机分为3块，实现语言自选 数据参见：groceries - groceries.csv 参数： 输出所有2阶、3阶频繁项 支持度统一设置为0.005 算法过程参考课件

实验步骤

任务一：

流程图

代码

代码总解释

```
import pandas as pd

from collections import defaultdict, Counter

from itertools import combinations

import math



def load_data(filepath):

    """加载数据，每行是一个交易"""
```

```
    df = pd.read_csv(filepath)
    transactions = []
    for idx, row in df.iterrows():
        # 跳过第一列（序号），收集非空商品
        items = [item for item in row[1:] if pd.notna(item) and str(item).strip()]
        if items:
            transactions.append(items)
    return transactions


def get_frequent_itemsets(transactions, min_support, k, prev_freq_itemsets=None):
    """
    获取k阶频繁项集
    transactions: 交易列表
    min_support: 最小支持度
    k: 项集大小
    prev_freq_itemsets: 上一阶的频繁项集（用于生成候选项集）
    """
    n_trans = len(transactions)
    min_count = math.ceil(min_support * n_trans)
    if k == 1:
        # 1阶：统计每个单品
        item_counts = Counter()
        for trans in transactions:
            for item in trans:
                item_counts[item] += 1
        # 筛选频繁项
        freq_itemsets = {}
```

```
        for item, count in item_counts.items():
            if count >= min_count:
                freq_itemsets[(item,)] = count
        return freq_itemsets
    else:
        # 从上一阶频繁项集生成候选项集
        candidates = {}
        prev_items = list(prev_freq_itemsets.keys())
        # 连接步：合并两个k-1阶项集
        for i in range(len(prev_items)):
            for j in range(i + 1, len(prev_items)):
                itemset1 = prev_items[i]
                itemset2 = prev_items[j]
                # 检查前k-2项是否相同
                if itemset1[:-1] == itemset2[:-1]:
                    new_itemset = tuple(sorted(set(itemset1) | set(itemset2)))
                    if len(new_itemset) == k:
                        # 剪枝步：检查所有k-1子集是否频繁
                        is_valid = True
                        for sub_itemset in combinations(new_itemset, k-1):
                            if sub_itemset not in prev_freq_itemsets:
                                is_valid = False
                                break
                        if is_valid:
                            candidates[new_itemset] = 0
        # 计数
        for trans in transactions:
```

```
            trans_set = set(trans)
            for itemset in candidates:
                if set(itemset).issubset(trans_set):
                    candidates[itemset] += 1
        # 筛选频繁项
        freq_itemsets = {}
        for itemset, count in candidates.items():
            if count >= min_count:
                freq_itemsets[itemset] = count
        return freq_itemsets


def apriori(transactions, min_support, max_k=3):
    """Apriori主函数"""
    n_trans = len(transactions)
    min_count = math.ceil(min_support * n_trans)
    print(f"总交易数: {n_trans}")
    print(f"最小支持度: {min_support}, 最小计数: {min_count}")
    print("=" * 60)
    all_freq_itemsets = {}
    # 1阶频繁项
    freq_1 = get_frequent_itemsets(transactions, min_support, 1)
    all_freq_itemsets.update(freq_1)
    # 2阶和3阶频繁项
    freq_prev = freq_1
    for k in range(2, max_k + 1):
        freq_k = get_frequent_itemsets(transactions, min_support, k, freq_prev)
        all_freq_itemsets.update(freq_k)
```

```
        if not freq_k:
            break
        freq_prev = freq_k
    # 分离2阶和3阶结果
    freq_2 = {itemset: count for itemset, count in all_freq_itemsets.items() if
len(itemset) == 2}
    freq_3 = {itemset: count for itemset, count in all_freq_itemsets.items() if
len(itemset) == 3}
    return freq_2, freq_3, all_freq_itemsets


def format_itemset(itemset):
    """格式化项集为字符串"""
    return ", ".join(itemset)


def main():
    # 加载数据
    print("加载数据中...")
    transactions = load_data("groceries - groceries.csv")
    print(f"加载完成，共 {len(transactions)} 条交易记录")
    # 设置参数
    min_support = 0.005
    # 运行Apriori算法
    print("\n运行Apriori算法...")
    freq_2, freq_3, _ = apriori(transactions, min_support, max_k=3)
    # 按支持度排序
    n_trans = len(transactions)
    # 排序2阶频繁项
    freq_2_sorted = sorted(freq_2.items(), key=lambda x: x[1], reverse=True)[:50]
```

```
    # 排序3阶频繁项
    freq_3_sorted = sorted(freq_3.items(), key=lambda x: x[1], reverse=True)[:50]
    # 输出2阶频繁项
    print("\n" + "=" * 60)
    print("前50项 2阶频繁项集 (支持度 >= {})".format(min_support))
    print("=" * 60)
    print(f"{'序号':<6} {'项集':<40} {'计数':<8} {'支持度':<10}")
    print("-" * 60)
    for i, (itemset, count) in enumerate(freq_2_sorted, 1):
        support = count / n_trans
        itemset_str = format_itemset(itemset)
        print(f"{i:<6} {itemset_str:<40} {count:<8} {support:.4f}")
    # 输出3阶频繁项
    print("\n" + "=" * 60)
    print("前50项 3阶频繁项集 (支持度 >= {})".format(min_support))
    print("=" * 60)
    print(f"{'序号':<6} {'项集':<50} {'计数':<8} {'支持度':<10}")
    print("-" * 70)
    for i, (itemset, count) in enumerate(freq_3_sorted, 1):
        support = count / n_trans
        itemset_str = format_itemset(itemset)
        print(f"{i:<6} {itemset_str:<50} {count:<8} {support:.4f}")


if __name__ == "__main__":
    main()
```

分块解释

加载并清洗 load_data

得到事务的集合T

```
def load_data(filepath):
    """加载数据，每行是一个交易"""
    df = pd.read_csv(filepath)
    transactions = []
    for idx, row in df.iterrows():
        # 跳过第一列（序号），收集非空商品
        items = [item for item in row[1:] if pd.notna(item) and str(item).strip()]
        if items:
            transactions.append(items)
    return transactions
```

生成k阶频繁项集 get_frequent_itemsets

生成候选集 → 剪枝 → 计数 → 筛选频繁项集

k=1时；生成一阶频繁项集

```
if k == 1:
        # 1阶：统计每个单品
        item_counts = Counter()
        for trans in transactions:
            for item in trans:
                item_counts[item] += 1 #为每一项目计数
        # 筛选频繁项
        freq_itemsets = {}
        for item, count in item_counts.items():
            if count >= min_count:
```

```
                freq_itemsets[(item,)] = count #大于最小支持度的留下
        return freq_itemsets
```

k≥2 时：连接 + 剪枝（Apriori 核心）

连接：合并两个k-1阶项集

```
 # 连接步：合并两个k-1阶项集
        for i in range(len(prev_items)):
            for j in range(i + 1, len(prev_items)):
                itemset1 = prev_items[i]
                itemset2 = prev_items[j]
                # 检查前k-2项是否相同
                if itemset1[:-1] == itemset2[:-1]:#k-2项相同
                    new_itemset = tuple(sorted(set(itemset1) | set(itemset2))) #
合并
```

剪枝

`candidates` 是一个**字典**：``

```
 # 剪枝步：检查所有k-1子集是否频繁
                    is_valid = True
                    for sub_itemset in combinations(new_itemset, k-1):
                        if sub_itemset not in prev_freq_itemsets:
                            is_valid = False
                            break
```

计数+筛选满足最小支持度的频繁项集

```
# 计数
        for trans in transactions:
            trans_set = set(trans)
            for itemset in candidates:
                if set(itemset).issubset(trans_set):
                    candidates[itemset] += 1
        # 筛选频繁项
        freq_itemsets = {}
        for itemset, count in candidates.items():
            if count >= min_count:
                freq_itemsets[itemset] = count
        return freq_itemsets
```

实验结果

![[Pasted image 20260426121044.png]] ![[Pasted image 20260426121059.png]] ![[Pasted image 20260426121111.png]] ![[Pasted image 20260426121137.png]]

任务二：

代码

```
package com.example;


import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
```

```
import org.apache.hadoop.mapreduce.Reducer;

import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;

import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;



import java.io.IOException;

import java.util.*;



public class AprioriMR {



    // ==================== L1 ====================

    public static class L1Mapper extends Mapper<Object, Text, Text, IntWritable> {

        private final static IntWritable one = new IntWritable(1);

        private Text word = new Text();



        @Override

        protected void map(Object key, Text value, Context context) throws
IOException, InterruptedException {

            String[] items = value.toString().split(",");

            for (String s : items) {

                String item = s.trim();

                if (!item.isEmpty()) {

                    word.set(item);

                    context.write(word, one);

                }

            }

        }

    }
```

```
    public static class L1Reducer extends Reducer<Text, IntWritable, Text,
IntWritable> {

        private int minCount;



        @Override

        protected void setup(Context context) {

            minCount = context.getConfiguration().getInt("min", 1);

        }



        @Override

        protected void reduce(Text key, Iterable<IntWritable> values, Context
context) throws IOException, InterruptedException {

            int sum = 0;

            for (IntWritable val : values) sum += val.get();

            if (sum >= minCount) context.write(key, new IntWritable(sum));

        }

    }



    // ==================== L2 ====================

    public static class L2Mapper extends Mapper<Object, Text, Text, IntWritable> {

        private final static IntWritable one = new IntWritable(1);

        private Text keyOut = new Text();

        private Set<String> l1Items = new HashSet<>();



        @Override

        protected void setup(Context context) {

            String l1 = context.getConfiguration().get("l1");

            if (l1 != null) l1Items.addAll(Arrays.asList(l1.split(",")));
```

```
        }


        @Override
        protected void map(Object key, Text value, Context context) throws
IOException, InterruptedException {
            Set<String> trans = new HashSet<>();
            for (String s : value.toString().split(",")) {
                String item = s.trim();
                if (!item.isEmpty() && l1Items.contains(item)) trans.add(item);
            }
            List<String> list = new ArrayList<>(trans);
            for (int i = 0; i < list.size(); i++) {
                for (int j = i + 1; j < list.size(); j++) {
                    keyOut.set(list.get(i) + "," + list.get(j));
                    context.write(keyOut, one);
                }
            }
        }
    }


    public static class L2Reducer extends Reducer<Text, IntWritable, Text,
IntWritable> {
        private int minCount;


        @Override
        protected void setup(Context context) {
            minCount = context.getConfiguration().getInt("min", 1);
        }
```

```
        @Override
        protected void reduce(Text key, Iterable<IntWritable> values, Context
context) throws IOException, InterruptedException {
            int sum = 0;
            for (IntWritable val : values) sum += val.get();
            if (sum >= minCount) context.write(key, new IntWritable(sum));
        }
    }

    // ==================== L3 ====================
    public static class L3Mapper extends Mapper<Object, Text, Text, IntWritable> {
        private final static IntWritable one = new IntWritable(1);
        private Text keyOut = new Text();

        @Override
        protected void map(Object key, Text value, Context context) throws
IOException, InterruptedException {
            Set<String> trans = new HashSet<>();
            for (String s : value.toString().split(",")) {
                String item = s.trim();
                if (!item.isEmpty()) trans.add(item);
            }
            List<String> list = new ArrayList<>(trans);
            for (int i = 0; i < list.size(); i++)
                for (int j = i + 1; j < list.size(); j++)
                    for (int k = j + 1; k < list.size(); k++) {
                        keyOut.set(list.get(i) + "," + list.get(j) + "," +
```

```
list.get(k));

                        context.write(keyOut, one);

                    }

        }

    }



    public static class L3Reducer extends Reducer<Text, IntWritable, Text,
IntWritable> {

        private int minCount;



        @Override

        protected void setup(Context context) {

            minCount = context.getConfiguration().getInt("min", 1);

        }



        @Override

        protected void reduce(Text key, Iterable<IntWritable> values, Context
context) throws IOException, InterruptedException {

            int sum = 0;

            for (IntWritable val : values) sum += val.get();

            if (sum >= minCount) context.write(key, new IntWritable(sum));

        }

    }



    // ==================== 主函数 ====================

    public static void main(String[] args) throws Exception {

        Configuration conf = new Configuration();

        final double MIN_SUPPORT = 0.005;
```

```
        // 自动计算最小出现次数
        int total = 9835; // groceries 数据集默认大小
        int minCount = (int) Math.ceil(total * MIN_SUPPORT);
        conf.setInt("min", minCount);

        // L1
        Job job1 = Job.getInstance(conf, "L1");
        job1.setJarByClass(AprioriMR.class);
        job1.setMapperClass(L1Mapper.class);
        job1.setReducerClass(L1Reducer.class);
        job1.setOutputKeyClass(Text.class);
        job1.setOutputValueClass(IntWritable.class);
        FileInputFormat.addInputPath(job1, new Path("/input/groceries.csv"));
        FileOutputFormat.setOutputPath(job1, new Path("/output/l1"));
        job1.waitForCompletion(true);

        // L2
        Job job2 = Job.getInstance(conf, "L2");
        job2.setJarByClass(AprioriMR.class);
        job2.setMapperClass(L2Mapper.class);
        job2.setReducerClass(L2Reducer.class);
        job2.setOutputKeyClass(Text.class);
        job2.setOutputValueClass(IntWritable.class);
        FileInputFormat.addInputPath(job2, new Path("/input/groceries.csv"));
        FileOutputFormat.setOutputPath(job2, new Path("/output/l2"));
        job2.waitForCompletion(true);
```

```
        // L3
        Job job3 = Job.getInstance(conf, "L3");
        job3.setJarByClass(AprioriMR.class);
        job3.setMapperClass(L3Mapper.class);
        job3.setReducerClass(L3Reducer.class);
        job3.setOutputKeyClass(Text.class);
        job3.setOutputValueClass(IntWritable.class);
        FileInputFormat.addInputPath(job3, new Path("/input/groceries.csv"));
        FileOutputFormat.setOutputPath(job3, new Path("/output/l3"));
        System.exit(job3.waitForCompletion(true) ? 0 : 1);
    }
}
```

局部代码分析

工具类：AprioriUtils

作用：提供Apriori算法所需的通用工具方法，包括候选集生成、项集与字符串的格式转换，简化核心代码逻辑。

```
// 工具类
class AprioriUtils {
    // 生成k阶候选集（核心方法：连接+剪枝）
    public static Set<Set<String>> generateCandidates(Set<Set<String>>
frequentItemsets, int k) {
        Set<Set<String>> candidates = new HashSet<>();
        List<Set<String>> itemsetList = new ArrayList<>(frequentItemsets);

        // 1. 连接操作：将两个k-1阶频繁项集合并，生成k阶候选集
        for (int i = 0; i < itemsetList.size(); i++) {
            for (int j = i + 1; j < itemsetList.size(); j++) {
                Set<String> itemset1 = itemsetList.get(i);
                Set<String> itemset2 = itemsetList.get(j);
                List&lt;String> list1 = new ArrayList<>(itemset1);
                List&lt;String> list2 = new ArrayList<>(itemset2);
                Collections.sort(list1);
                Collections.sort(list2);
```

```
                    // 判断两个k-1阶项集是否可连接（前k-2个元素相同）
                    boolean canJoin = true;
                    for (int m = 0; m < k - 2; m++) {
                        if (!list1.get(m).equals(list2.get(m))) {
                            canJoin = false;
                            break;
                        }
                    }
                    if (canJoin) {
                        Set<String> newItemset = new HashSet<>(itemset1);
                        newItemset.addAll(itemset2);
                        // 确保合并后是k阶项集
                        if (newItemset.size() == k) {
                            // 2. 剪枝操作：删除子集非频繁的候选集
                            boolean prune = false;
                            List&lt;String> newList = new ArrayList<>(newItemset);
                            for (int m = 0; m < newList.size(); m++) {
                                Set&lt;String> subset = new HashSet<>(newList);
                                subset.remove(newList.get(m));
                                if (!frequentItemsets.contains(subset)) {
                                    prune = true;
                                    break;
                                }
                            }
                            if (!prune) {
                                candidates.add(newItemset);
                            }
                        }
                    }
                }
            }
            return candidates;
        }

        // 项集转换为字符串（方便MapReduce输出，格式：商品1,商品2,...）
        public static String itemsetToString(Set<String> itemset) {
            List<String> list = new ArrayList<>(itemset);
            Collections.sort(list);
            return String.join(",", list);
        }

        // 字符串转换为项集（方便读取MapReduce输出结果）
        public static Set<String> stringToItemset(String str) {
            return new HashSet<>(Arrays.asList(str.split(",")));
        }
    }
```

1阶频繁项集（L1）：L1Mapper + L1Reducer

L1Mapper

```
class L1Mapper extends Mapper<Object, Text, Text, IntWritable> {
    private final static IntWritable one = new IntWritable(1);
    private Text item = new Text();

    @Override
    protected void map(Object key, Text value, Context context) throws
IOException, InterruptedException {
        // 读取一行交易数据，按逗号切分成单个商品
        String[] items = value.toString().split(",");
        for (String s : items) {
            String trimItem = s.trim();
            // 过滤空值，避免无效数据
            if (!trimItem.isEmpty()) {
                item.set(trimItem);
                // 输出键值对：<商品名，1>，表示该商品在本次交易中出现1次
                context.write(item, one);
            }
        }
    }
}
```

L1Reducer

```
class L1Reducer extends Reducer<Text, IntWritable, Text, IntWritable> {
    private int minCount; // 最小支持次数（总交易数 * 最小支持度0.005）

    @Override
    protected void setup(Context context) {
        // 从配置中获取最小支持次数
        minCount = context.getConfiguration().getInt("minCount", 1);
    }

    @Override
    protected void reduce(Text key, Iterable<IntWritable> values, Context context)
throws IOException, InterruptedException {
        // 统计当前商品的总出现次数（聚合Map阶段的输出）
        int sum = 0;
        for (IntWritable val : values) sum += val.get();
        // 筛选出满足最小支持次数的商品（即1阶频繁项集）
        if (sum >= minCount) context.write(key, new IntWritable(sum));
    }
}
```

2阶频繁项集（L2）：L2Mapper + L2Reducer

L2Mapper

```
public static class L2Mapper extends Mapper<Object, Text, Text, IntWritable> {

        private final static IntWritable one = new IntWritable(1);

        private Text keyOut = new Text();

        private Set<String> l1Items = new HashSet<>();



        @Override

        protected void setup(Context context) {

            String l1 = context.getConfiguration().get("l1");

            if (l1 != null) l1Items.addAll(Arrays.asList(l1.split(",")));

        }



        @Override

        protected void map(Object key, Text value, Context context) throws
IOException, InterruptedException {

            Set<String> trans = new HashSet<>();

            for (String s : value.toString().split(",")) {

                String item = s.trim();

                if (!item.isEmpty() && l1Items.contains(item)) trans.add(item);

            }

            List<String> list = new ArrayList<>(trans);

            for (int i = 0; i < list.size(); i++) {

                for (int j = i + 1; j < list.size(); j++) {

                    keyOut.set(list.get(i) + "," + list.get(j));

                    context.write(keyOut, one);

                }

            }

        }
```

```
    }
```

L2Reducer

```
public static class L2Reducer extends Reducer<Text, IntWritable, Text,
IntWritable> {

        private int minCount;



        @Override

        protected void setup(Context context) {

            minCount = context.getConfiguration().getInt("min", 1);

        }



        @Override

        protected void reduce(Text key, Iterable<IntWritable> values, Context
context) throws IOException, InterruptedException {

            int sum = 0;

            for (IntWritable val : values) sum += val.get();

            if (sum >= minCount) context.write(key, new IntWritable(sum));

        }

    }
```

3阶频繁项集（L3）：L3Mapper + L3Reducer

L3Mapper

```
class L3Mapper extends Mapper<Object, Text, Text, IntWritable> {
    private final static IntWritable one = new IntWritable(1);
    private Text itemsetText = new Text();
    private Set&lt;Set&lt;String>> L2 = new HashSet<>(); // 存储2阶频繁项集

    @Override
    protected void setup(Context context) {
```

```
        // 从配置中读取L2的结果，转换为项集格式
        String l2Str = context.getConfiguration().get("L2");
        if (l2Str != null) {
            String[] itemsets = l2Str.split(";");
            for (String s : itemsets) {
                if (!s.isBlank()) L2.add(AprioriUtils.stringToItemset(s.trim()));
            }
        }
    }

    @Override
    protected void map(Object key, Text value, Context context) throws
IOException, InterruptedException {
        // 读取一行交易数据，获取所有商品
        String[] items = value.toString().split(",");
        Set&lt;String> trans = new HashSet<>();
        for (String s : items) {
            String t = s.trim();
            if (!t.isEmpty()) trans.add(t);
        }
        // 生成3阶候选集（调用工具类方法，连接+剪枝）
        Set<Set<String>> C3 = AprioriUtils.generateCandidates(L2, 3);
        // 判断当前交易是否包含候选集，包含则计数
        for (Set<String> cand : C3) {
            if (trans.containsAll(cand)) {
                itemsetText.set(AprioriUtils.itemsetToString(cand));
                // 输出键值对：<"商品A,商品B,商品C", 1>
                context.write(itemsetText, one);
            }
        }
    }
}
```

L3Mapper

```
class L3Reducer extends Reducer<Text, IntWritable, Text, IntWritable> {
    private int minCount;

    @Override
    protected void setup(Context context) {
        minCount = context.getConfiguration().getInt("minCount", 1);
    }

    @Override
    protected void reduce(Text key, Iterable<IntWritable> values, Context context)
throws IOException, InterruptedException {
        // 统计每个3阶组合的出现次数
        int sum = 0;
        for (IntWritable val : values) sum += val.get();
        // 筛选出满足最小支持次数的3阶频繁项集
        if (sum >= minCount) context.write(key, new IntWritable(sum));
```

```
    }
}
```

任务三：

流程图

![[Pasted image 20260426221818.png]]

总代码

```
import csv

import math

import random

from collections import Counter

from itertools import combinations



# ------------------------------

# 1. 数据加载

# ------------------------------

def load_transactions(filepath):

    """加载交易数据，每行格式：数量，商品1，商品2，..."""

    transactions = []

    with open(filepath, 'r', encoding='utf-8') as f:

        reader = csv.reader(f)

        # 跳过表头（第一行）

        header = next(reader)

        for row in reader:

            if not row:

                continue

            # 第一列为商品数量（可选，跳过），其余为非空商品
```

```
            items = []
            for item in row[1:]:
                item = item.strip()
                if item and item != '':
                    items.append(item)
            if items:
                transactions.append(items)
    return transactions


# ------------------------------
# 2. Apriori 核心函数（支持给定最小计数值）
# ------------------------------
def apriori_freq_itemsets(transactions, min_count, max_k=3):
    """
    在给定的交易列表上运行 Apriori，返回所有长度 <= max_k 的频繁项集
    transactions : list of list
    min_count    : 绝对最小支持度计数
    max_k        : 最大项集长度
    返回值 : dict {itemset: count}
    """
    n = len(transactions)
    if n == 0:
        return {}

    # 1-频繁项集
    item_cnt = Counter()
    for trans in transactions:
```

```
        for item in trans:
            item_cnt[item] += 1
    freq = { (item,): cnt for item, cnt in item_cnt.items() if cnt >= min_count }
    all_freq = freq.copy()


    prev_freq = freq
    for k in range(2, max_k + 1):
        # 生成候选
        candidates = {}
        prev_items = list(prev_freq.keys())
        for i in range(len(prev_items)):
            for j in range(i+1, len(prev_items)):
                set1 = prev_items[i]
                set2 = prev_items[j]
                if set1[:-1] == set2[:-1]:
                    new_set = tuple(sorted(set(set1) | set(set2)))
                    if len(new_set) == k:
                        # 剪枝：所有 k-1 子集必须频繁
                        valid = True
                        for sub in combinations(new_set, k-1):
                            if sub not in prev_freq:
                                valid = False
                                break
                        if valid:
                            candidates[new_set] = 0
        if not candidates:
            break
```

```
        # 计数
        for trans in transactions:
            trans_set = set(trans)
            for cand in candidates:
                if set(cand).issubset(trans_set):
                    candidates[cand] += 1

        # 筛选频繁项
        freq_k = {cand: cnt for cand, cnt in candidates.items() if cnt >=
min_count}
        all_freq.update(freq_k)
        prev_freq = freq_k
        if not prev_freq:
            break

    return all_freq

# ------------------------------
# 3. SON 算法主函数
# ------------------------------
def son_algorithm(transactions, support, num_blocks=3, max_k=3):
    """
    SON 算法（随机分块 + 局部挖掘 + 全局验证）
    support : 全局相对支持度 (0~1)
    returns: 最终频繁项集字典 {itemset: count}
    """
    total_trans = len(transactions)
```

```
global_min_count = math.ceil(support * total_trans)


# 随机打乱并分成 num_blocks 块
shuffled = transactions.copy()
random.shuffle(shuffled)
block_size = math.ceil(total_trans / num_blocks)
blocks = [shuffled[i:i+block_size] for i in range(0, total_trans, block_size)]
# 确保恰好 num_blocks 块（最后一块可能略小）
while len(blocks) < num_blocks:
    blocks.append([])


print(f"全局交易数: {total_trans}, 全局最小计数: {global_min_count}")
print(f"分 {num_blocks} 块, 各块大小: {[len(b) for b in blocks]}")


# 步骤1：每个块上挖掘局部频繁项集
local_candidates = set()
for i, block in enumerate(blocks):
    if not block:
        continue
    block_min_count = math.ceil(support * len(block))
    print(f"  块 {i+1}: 大小={len(block)}, 局部最小计数={block_min_count}")
    freq_local = apriori_freq_itemsets(block, block_min_count, max_k=max_k)
    for itemset in freq_local.keys():
        local_candidates.add(itemset)
print(f"局部候选集大小: {len(local_candidates)}")


# 步骤2：全局扫描验证候选集
```

```
    candidate_counts = {cand: 0 for cand in local_candidates}
    for trans in transactions:
        trans_set = set(trans)
        for cand in candidate_counts:
            if set(cand).issubset(trans_set):
                candidate_counts[cand] += 1

    # 步骤3：保留全局频繁的项集
    final_freq = {cand: cnt for cand, cnt in candidate_counts.items()
                  if cnt >= global_min_count}
    print(f"全局频繁项集数量: {len(final_freq)}")
    return final_freq


# ------------------------------
# 4. 输出格式化
# ------------------------------
def format_itemset(itemset):
    return " + ".join(itemset)


def print_top_k(freq_dict, k, title, total_trans):
    """输出前k个项集（按支持度降序）"""
    sorted_items = sorted(freq_dict.items(), key=lambda x: x[1], reverse=True)[:k]
    print("\n" + "="*70)
    print(f"{title} (支持度阈值 = {min_support})")
    print("="*70)
    print(f"{'序号':<6} {'项集':<50} {'计数':<8} {'支持度':<10}")
    print("-"*70)
```

```
    for i, (itemset, cnt) in enumerate(sorted_items, 1):
        support = cnt / total_trans
        itemset_str = format_itemset(itemset)
        print(f"{i:<6} {itemset_str:<50} {cnt:<8} {support:.4f}")


# ------------------------------
# 5. 主程序
# ------------------------------
if __name__ == "__main__":
    # 固定随机种子，保证结果可复现（可选）
    random.seed(42)

    # 加载数据
    filepath = "groceries - groceries.csv"
    print("加载数据...")
    transactions = load_transactions(filepath)
    print(f"加载完成，共 {len(transactions)} 条交易")

    # SON 参数
    min_support = 0.005
    num_blocks = 3
    max_k = 3

    # 运行 SON 算法
    print("\n开始运行 SON 算法...")
    final_freq = son_algorithm(transactions, min_support, num_blocks, max_k)
```

```
    # 分离 2 阶和 3 阶项集

    freq_2 = {itemset: cnt for itemset, cnt in final_freq.items() if len(itemset)
== 2}

    freq_3 = {itemset: cnt for itemset, cnt in final_freq.items() if len(itemset)
== 3}



    total = len(transactions)

    print_top_k(freq_2, 50, "前50项 2阶频繁项集 (SON算法)", total)

    print_top_k(freq_3, 50, "前50项 3阶频繁项集 (SON算法)", total)
```

代码分步详解

模块 1：数据加载 `load_transactions(filepath)`

```
def load_transactions(filepath):

    """加载交易数据，每行格式：数量，商品1，商品2，..."""

    transactions = []

    with open(filepath, 'r', encoding='utf-8') as f:

        reader = csv.reader(f)

        # 跳过表头（第一行）

        header = next(reader)

        for row in reader:

            if not row:

                continue

            # 第一列为商品数量（可选，跳过），其余为非空商品

            items = []

            for item in row[1:]:

                item = item.strip()

                if item and item != '':
```

```
                    items.append(item)
            if items:
                transactions.append(items)
    return transactions
```

模块 2：Apriori 核心函数

正常的apriori算法 先计算k1 然后计算k2,k3

```
def apriori_freq_itemsets(transactions, min_count, max_k=3):
    """
    在给定的交易列表上运行 Apriori，返回所有长度 <= max_k 的频繁项集
    transactions : list of list
    min_count    : 绝对最小支持度计数
    max_k        : 最大项集长度
    返回值 : dict {itemset: count}
    """
    n = len(transactions)
    if n == 0:
        return {}

    # 1-频繁项集
    item_cnt = Counter()
    for trans in transactions:
        for item in trans:
            item_cnt[item] += 1
    freq = { (item,): cnt for item, cnt in item_cnt.items() if cnt >= min_count }
    all_freq = freq.copy()
```

```
    prev_freq = freq
    for k in range(2, max_k + 1):
        # 生成候选
        candidates = {}
        prev_items = list(prev_freq.keys())
        for i in range(len(prev_items)):
            for j in range(i+1, len(prev_items)):
                set1 = prev_items[i]
                set2 = prev_items[j]
                if set1[:-1] == set2[:-1]:
                    new_set = tuple(sorted(set(set1) | set(set2)))
                    if len(new_set) == k:
                        # 剪枝：所有 k-1 子集必须频繁
                        valid = True
                        for sub in combinations(new_set, k-1):
                            if sub not in prev_freq:
                                valid = False
                                break
                        if valid:
                            candidates[new_set] = 0
        if not candidates:
            break

        # 计数
        for trans in transactions:
            trans_set = set(trans)
            for cand in candidates:
```

```
                if set(cand).issubset(trans_set):
                    candidates[cand] += 1

        # 筛选频繁项
        freq_k = {cand: cnt for cand, cnt in candidates.items() if cnt >= min_count}
        all_freq.update(freq_k)
        prev_freq = freq_k
        if not prev_freq:
            break

    return all_freq
```

模块 3：SON 算法主函数

步骤1：随机分块

```
 # 随机打乱并分成 num_blocks 块
    shuffled = transactions.copy()
    random.shuffle(shuffled)
    block_size = math.ceil(total_trans / num_blocks)
    blocks = [shuffled[i:i+block_size] for i in range(0, total_trans, block_size)]
    # 确保恰好 num_blocks 块（最后一块可能略小）
    while len(blocks) < num_blocks:
        blocks.append([])
```

步骤 2：局部候选生成

```
block_min_count = math.ceil(support * len(block))
```

```
        print(f"  块 {i+1}: 大小={len(block)}, 局部最小计数={block_min_count}")
        freq_local = apriori_freq_itemsets(block, block_min_count, max_k=max_k)
        for itemset in freq_local.keys():
            local_candidates.add(itemset)
```

步骤3：全局验证

防止伪正例

```
candidate_counts = {cand: 0 for cand in local_candidates}
    for trans in transactions:
        trans_set = set(trans)
        for cand in candidate_counts:
            if set(cand).issubset(trans_set):
                candidate_counts[cand] += 1
```

步骤 4：筛选全局频繁项集

```
    final_freq = {cand: cnt for cand, cnt in candidate_counts.items()
                  if cnt >= global_min_count}
    print(f"全局频繁项集数量: {len(final_freq)}")
    return final_freq
```

实验结果

![[Pasted image 20260426235508.png]] ![[Pasted image 20260426235621.png]] ![[Pasted image 20260426235556.png]]

实验总结

Apriori算法是用来求关联规则的一种算法，不过本实验倒是没有设计置信度的计算，其实并没有求出关联规则，只是求了频繁集